RUBBER STAMP: GEMMA 4 SCORES ITS OWN FAILING PATCHES AS SURE TO PASS

Asked to check SWE-bench patches without knowing who wrote them, Gemma 4 31B gives most of its own failing patches the top score, and outside judges do not. A second judge lets an agent built on Gemma pick out the patches it can submit without review.

ABSTRACT

A coding agent has to decide when its patch is done: submit it, try again, or ask a person. That decision is only as good as the agent's check of its own work. We test how well Gemma 4 31B checks its own patches. Nine open models scored the chance that a patch passes the hidden tests of its SWE-bench Verified issue, without being told who wrote it, and we compared each judge with outside judges scoring the same patches.

On 193 held-out issues, Gemma gives a score of 100 to 61% of its own failing patches and to 35% of other models' failing patches. Outside judges scoring the same patches do not share this extra trust, and it is absent on passing patches. The pattern repeats on a second issue set and when Gemma's patches come from the same agent program as everyone else's. Gemma also ranks its own patches less accurately than outside judges do, but about half of our first estimate of that gap came from a difference between agent programs. Five other models show no gap.

The cost to an agent is concrete. If it submits without review only the quarter of its patches that its check trusts most, 61% of them pass when Gemma checks itself and 84% when GLM-4.5 does. We release the patches, every judge answer and the analysis code, and give a three-step audit for any agent's self-check.

1. INTRODUCTION

Coding agents end a task by deciding that their patch is done. Many agents and agent papers ask the same model to judge its own work, for example to decide when to stop or whether a patch can go out without review. That works only if the model judges its own patches as well as it judges anyone else's.

We measure this for Gemma 4 31B, the model of this competition. Each judge sees an issue and one patch, writes a short explanation, and gives a 0 to 100 score for the chance that the patch passes the hidden tests. The prompt never names the author. A model can score its own patches poorly simply because they are hard, so we compare it with outside judges: models from other families scoring the same patches.

Our first look used public Gemma patches from four agent programs (harnesses), while every other writer's patches came from one harness, mini-SWE-agent. To remove that difference, we ran Gemma 4 31B in mini-SWE-agent ourselves, on the same 50 issues and on 200 held-out issues, under a plan committed before the runs.

The paper tells one story. Gemma rubber-stamps its own failing patches (Section 3.1). This also costs it ranking accuracy, though less than we first measured (Section 3.2), and no other model we tested shows the gap (Section 3.3). A second judge fixes much of it for an agent deciding what to submit (Section 3.4). Section 4 turns the method into an audit that any team can run on its own agent.

2. SETUP

Patch sets. All issues come from SWE-bench Verified (Jimenez et al., 2023). Other writers' patches are public submissions (SWE-bench experiments) made with mini-SWE-agent by GLM-4.5, gpt-oss-120b, Kimi-K2, Llama-4-Maverick, Qwen3-Coder and, on the Mini issues, Devstral, with their

official labels. Test files were removed from every patch, and patches over 30,000 characters were dropped.

Patch set	Issues	Gemma's patches	Scored by all nine judges (Gemma's / others')
Mini issues, mixed harness	50 (django, sphinx)	public runs in four harnesses	150 / 183
Mini issues, matched harness	the same 50	ours, mini-SWE-agent	42 / 183
Held-out issues	193, from 11 repositories	ours, mini-SWE-agent	180 / 777

The Mini issues are SWE-bench Verified Mini (MariusHobbhahn). Their mixed-harness Gemma patches come from public third-party runs (n-pelleriti) in SWE-agent (Yang et al., 2024), openclaw, opencode and zeroclaw. The held-out issues are a fixed random sample of 200 SWE-bench Verified issues outside the Mini issues. For our own runs, the 4-bit competition checkpoint of Gemma 4 31B made one attempt per issue in mini-SWE-agent v1.7.0 (SWE-agent) with its default settings and temperature 0. We graded these patches with SWE-bench's grading code and kept an issue only if its reference patch passes in our environment, which dropped 7 of the 200 held-out issues. Gemma solved 98 of the 193 kept issues.

Judges. Nine models judged every patch: Gemma 4 31B, gpt-oss-20b, Qwen3.5-9B, GLM-4.7-Flash, and the five writers above other than Devstral. Calls went through OpenRouter at temperature 0 with reasoning switched off, except gpt-oss, which always reasons and ran at low effort. An answer without a score was asked once more with a larger output limit, a step we call a re-ask. Analyses use the patches that all nine judges scored.

Measures.

- AUROC: the chance that a judge scores a random passing patch above a random failing one (ties count half).
- Self-judging gap: how much a model's AUROC drops from other writers' patches to its own, minus the same drop for outside judges.
- Leniency: a judge's score on a patch minus the outside judges' mean score on that patch, compared between a model's own failing patches and other writers' failing patches.
- Rubber-stamp rate: the share of a judge's own failing patches that it scores 100.

Intervals are 95% intervals from 2,000 bootstrap resamples of issues.

Plans. Every test had a written plan with its reading rule, committed before its data was computed. Two changes came after the plan for our own patch sets, both before any of their results were read: we added the leniency measure, and we raised the re-ask limit from 3,000 to 8,000 tokens because many answers ran out before the score. Analyses chosen after we saw a result are marked exploratory.

3. RESULTS

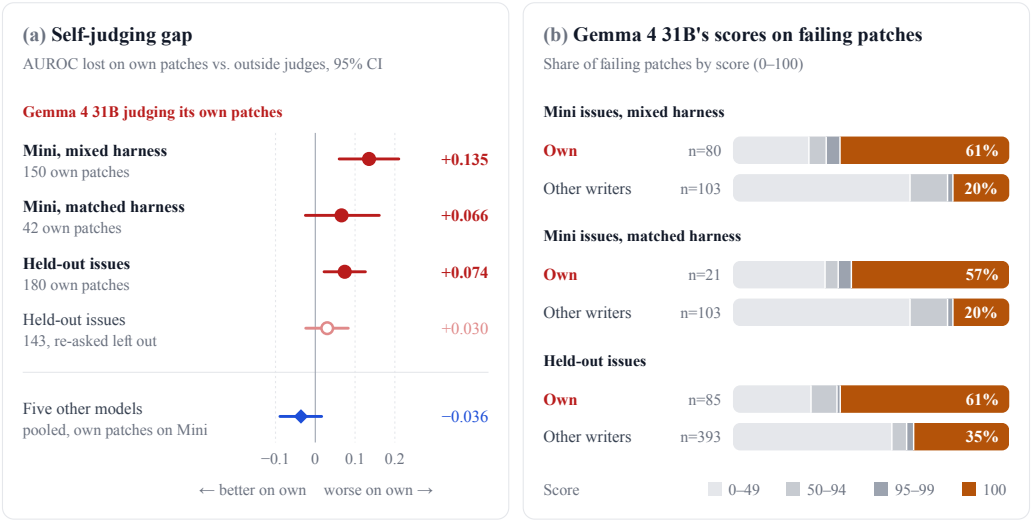


Figure 1: (a) Gemma 4 31B's self-judging gap on each patch set, with 95% intervals. The open circle leaves out every held-out patch that needed a re-ask (exploratory). The diamond is the pooled gap of five other models judging their own patches. (b) Gemma's scores on failing patches, its own and other writers', per patch set.

3.1 GEMMA RUBBER-STAMPS ITS OWN FAILING PATCHES

Patch set	Own failing patches scored 100	Others' failing patches scored 100	Leniency on failing patches [95%]
Mini issues, mixed harness	61% (49 of 80)	20% (of 103)	+20.9 [11.5, 30.2]
Mini issues, matched harness	57% (12 of 21)	20% (of 103)	+11.7 [−1.4, 24.2]
Held-out issues	61% (52 of 85)	35% (of 393)	+16.6 [10.3, 23.1]

Gemma's rubber-stamp rate stays near 60% whatever the issues or the harness. Some of this comes from the patches themselves, since some outside judges also give Gemma's failing patches a perfect score more often than other writers'. Leniency removes that part by comparing judges on the same patch. On the held-out issues, Gemma scores its own failing patches 16.6 points above the outside judges, more than it does for other writers' failing patches. On passing patches there is no such excess (+3.4 [−2.0, 8.0]), so the extra trust goes to exactly the patches Gemma should reject.

This result does not depend on the larger re-ask limit: leaving out every patch that needed a re-ask, the held-out leniency is still +11.1 [3.8, 18.2] (exploratory). Checks on the mixed-harness patches also rule out patch size, missing reasoning and unsolvable issues as explanations (Supplement, Section D).

3.2 THE RANKING GAP IS REAL BUT SMALLER THAN FIRST MEASURED

Patch set	Gemma's AUROC, own patches	Gemma's AUROC, others'	Self-judging gap [95%]
Mini issues, mixed harness	0.636	0.833	+0.135 [0.060, 0.210]
Mini issues, matched harness	0.696	0.833	+0.066 [−0.025, 0.161]
Held-out issues	0.622	0.751	+0.074 [0.022, 0.126]

Ties at 100 hide which of Gemma's patches fail, so rubber-stamping should also hurt its ranking of its own patches. On the same 50 issues, switching Gemma's patches to the matched harness halved

the gap, while the outside judges' drop on Gemma's patches stayed about the same. So about half of the first gap went with the harness that wrote the patches.

On the held-out issues, the gap meets the reading rule we fixed in advance, with little margin. It also depends on the larger re-ask limit: leaving out every patch that needed a re-ask gives +0.030 [−0.024, 0.083] (exploratory). We therefore treat the ranking gap as suggestive and the leniency in Section 3.1 as the firm result.

3.3 Only Gemma shows the gap

On the Mini issues, five of the other writers judged their own mini-SWE-agent patches, so the harness is matched for them by construction. Pooled over the five, the gap is −0.036 [−0.089, 0.016], and none of them shows a gap alone (Supplement, Section E). Their leniency is less clear, because each wrote only 14 to 24 failing patches.

3.4 What a second judge buys an agent

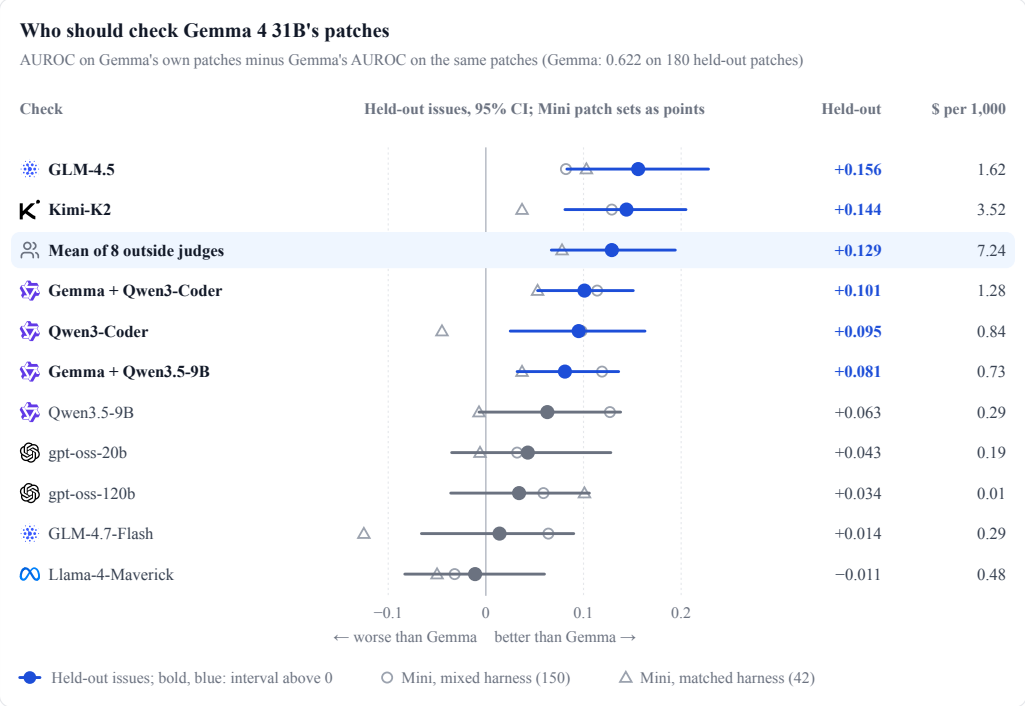


Figure 2: Each check's AUROC on Gemma's own patches minus Gemma's AUROC on the same patches. Lines are 95% intervals on the 180 held-out patches; hollow points are the two Mini patch sets. "Gemma + X" averages the two judges' ranks of each patch. Cost is the OpenRouter price per 1,000 patches in US dollars.

Consider an agent that submits some of its patches without review and sends the rest to a person. The table shows the share of submitted held-out patches that pass if the agent submits the quarter or the half of its patches that its check ranks highest. Without any check, 53% pass.

Check	Submit top quarter: share passing [gain over Gemma, 95%, points]	Submit top half: share passing [same]
Gemma's own score	61%	61%
Gemma + Qwen3.5-9B	76% [+6.7, +23.3]	68% [+1.5, +12.0]
Kimi-K2	77% [+9.6, +22.5]	72% [+5.9, +17.6]
GLM-4.5	84% [+12.7, +32.0]	74% [+6.8, +18.3]

Gemma's own score cannot make the agent more selective. It gives 100 to 74% of its patches, so the top quarter and the top half are random draws from that tied group, and 61% of them pass either way. A second judge breaks the tie. GLM-4.5 and Kimi-K2 rank Gemma's patches about as well as Gemma ranks other writers' patches (Figure 2). Qwen3.5-9B's gain alone is within the margin of error, but averaged with Gemma's ranks, at two cheap calls per patch, it is a clear gain.

These checks were chosen after we had seen their AUROCs, so this comparison is exploratory. We cannot tell whether the outside judges help because they come from another family or because they judge better in general.

4. Auditing an agent's self-check

The method carries over to any model and benchmark. A team that wants to trust its agent's own verdicts can run three steps on patches with known outcomes:

1. **Compare with outside judges on the same patches.** A model may rank its own patches poorly only because they are hard. Subtracting the outside judges' drop on the same patches removes that.
2. **Match the harness.** Patches from different harnesses differ in more than their author. In our data, half of the first gap came from this difference.
3. **Measure leniency on failing patches, patch by patch.** Ranking measures mix leniency with issue difficulty and ties. Comparing judges on the same patch isolates the part a team can act on: patches the model passes that it should reject.

For an agent built on Gemma, the advice is to let a second model decide which patches go out without review: when submitting, opening a pull request or handing work to a person. It does not matter for choosing among several attempts at one issue, where Gemma picks the passing patch about as often as outside judges do (Supplement, Section D). Gemma and Qwen3.5-9B are both open models, which fits agents that run offline, but we ran them through an API and did not test the pair locally.

5. Related work

Judges that favor their own outputs have been studied mostly on text and short programs. Panickssery et al. (2024) link the strength of this self-preference to how well a model recognizes its own summaries. Wataoka et al. (2024) relate it to the perplexity of the preferred response. Roytburg et al. (2026) compare a judge's own failing answers with other models' failing answers to the same question, and much of the measured self-preference goes away. Our leniency measure also compares judges on the same patch. Chen et al. (2025) find that stronger judges more often prefer their own wrong answer to a correct one.

Two studies are closest to ours. Pombal et al. (2026) check programs from twelve models against unit-test rubrics and find that judges favor their own outputs even with objective rubrics. On LiveCodeBench, Gemma 3 12B and 27B do not: their ratios are 0.97 and 1.03, where 1 means no self-preference. Khullar et al. (2026) find that a model rates a SWE-bench patch as more correct when it is framed as the model's own earlier answer. When the patch is shown in a user turn, their ratings separate passing from failing patches with AUROC 0.99, for one unnamed model and with no comparison to other writers' patches.

Our setting differs in three ways. The patches come from full agent runs, the judge sees no authorship or framing cue, and we measure ranking accuracy against hidden tests with outside judges as the control. In this setting five of six models judge their own patches as well as outside judges do, while Gemma 4 does not, unlike the Gemma 3 models Pombal et al. tested.

6. Limitations

- **What transfers.** The audit method applies to any model and benchmark. The finding is about Gemma 4 31B on SWE-bench Verified. We do not know which part of its training causes it, or whether Gemma recognizes its own patches or shares the mistakes in them.
- **Writer and judge precision.** Our Gemma patches come from the 4-bit competition checkpoint, while Gemma as judge is the hosted full-precision model.
- **Labels from two sources.** Other writers' labels are SWE-bench's official ones, while we graded our Gemma patches ourselves. On 30 issues, 26 other-writer patches got their official label in our environment; the misses came from rebuilt images and environment faults.

- **Changes after the plan.** The held-out ranking gap depends on the larger re-ask limit; the leniency does not.
- **Size.** One attempt per issue in our own runs, and only 42 matched-harness patches on the Mini issues.

7. Data, code and supplement

All data are public, and no competition data was sent to an outside service. The repository (https://github.com/Vein05/gemma4-paper-track) holds the code, prompts, plans with their commit dates, every judge answer, Gemma's agent runs and their labels, and the script behind every number. The supplement (`rubber-stamp-supplement.pdf`) holds the exact prompt, grading checks, re-ask counts, per-judge tables, the mechanism checks, all submit-without-review rates and the cost of each run (12.36 US dollars of API calls in all).

References

- Wei-Lin Chen, Zhepei Wei, Xinyu Zhu, Shi Feng and Yu Meng. 2025. *Do LLM Evaluators Prefer Themselves for a Reason?* https://arxiv.org/abs/2504.03846.
- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press and Karthik Narasimhan. 2023. *SWE-bench: Can Language Models Resolve Real-World GitHub Issues?* https://arxiv.org/abs/2310.06770. ICLR 2024 (arXiv comment, https://openreview.net/forum?id=VTF8yNQM66).
- Dipika Khullar, Jack Hopkins, Rowan Wang and Fabien Roger. 2026. *Self-Attribution Bias: When AI Monitors Go Easy on Themselves*. https://arxiv.org/abs/2603.04582.
- MariusHobbhahn. *swe-bench-verified-mini*. https://huggingface.co/datasets/MariusHobbhahn/swe-bench-verified-mini. Hugging Face dataset; revision b316c349; accessed 2026-10-05.
- n-pelleriti. *AlphaDiana Gemma 4 agent-run datasets*. https://huggingface.co/n-pelleriti. Six Hugging Face datasets; accessed 2026-10-05.
- Arjun Panickssery, Samuel R. Bowman and Shi Feng. 2024. *LLM Evaluators Recognize and Favor Their Own Generations*. https://arxiv.org/abs/2404.13076.
- José Pombal, Ricardo Rei and André F. T. Martins. 2026. *Self-Preference Bias in Rubric-Based Evaluation of Large Language Models*. https://arxiv.org/abs/2604.06996.
- Dani Roytburg, Matthew Bozoukov, Matthew Nguyen, Jou Barzdukas, Mackenzie Puig-Hall and Narmeen Oozeer. 2026. *Are LLM Evaluators Really Narcissists? Sanity Checking Self-Preference Evaluations*. https://arxiv.org/abs/2601.22548. ICML 2026 Main.
- SWE-agent. *mini-SWE-agent*. https://github.com/SWE-agent/mini-swe-agent. GitHub repository; accessed 2026-10-05.
- SWE-bench. *SWE-bench experiments*. https://github.com/SWE-bench/experiments. GitHub repository; patches from the swe-bench-submissions bucket; revision 40f164d5; accessed 2026-10-05.
- Koki Wataoka, Tsubasa Takahashi and Ryokan Ri. 2024. *Self-Preference Bias in LLM-as-a-Judge*. https://arxiv.org/abs/2410.21819. Accepted at NeurIPS 2024 Safe Generative AI Workshop.
- John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan and Ofir Press. 2024. *SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering*. https://arxiv.org/abs/2405.15793.